

PRIVATE SHIELD: A Privacy-Preserving Federated Intelligence System for IoT Threat Detection

Md. Murad Ul Momin | Rima Saha | Sakia Tabassum Anonna | Ashraful Hossain Riyan

Group 06 | CSE 499A | Section 15 | Faculty: Mohammad Shifat-E-Rabbi

Abstract Internet of Things (IoT) environments generate distributed, heterogeneous, and privacy-sensitive traffic while many edge devices operate with limited memory, processing power, and built-in security. Conventional centralized intrusion detection simplifies global analysis but requires large volumes of raw telemetry to leave the local network, increasing privacy exposure, bandwidth use, and dependence on a central service. PRIVATE SHIELD is an academic prototype that studies a unified alternative based on federated learning. The system keeps local traffic at simulated IoT clients, exchanges model parameters rather than raw records, and combines privacy protection, robust aggregation, client-reliability checks, anomaly detection, and an operator-facing security dashboard. The project was designed from a literature review that identified five recurring limitations in existing work: gradient leakage, poisoning of model updates, free-rider participation, non-IID client data, and weak operational explainability. Current implementation work includes a basic SecureDyn-FL federated IDS prototype, a FedMADE dynamic aggregation baseline, a Weight Evolving Frequency (WEF) defense against free riders, and a FedMSE semi-supervised anomaly-detection baseline. The report describes the motivation, architecture, methodology, implementation progress, threat model, evaluation plan, limitations, and future integration path. Exact paper-reported performance values are not presented as project results because the current experiments use selected data portions and the team's available computing environment.

Index Terms—Federated learning, Internet of Things, intrusion detection, differential privacy, robust aggregation, free-rider defense, non-IID data, anomaly detection, explainable security.

I. INTRODUCTION

IoT devices are now embedded in homes, healthcare systems, transportation, surveillance, industrial monitoring, and smart infrastructure. Their wide deployment creates a large attack surface because many devices operate with low-power processors, limited memory, outdated firmware, weak authentication, or minimal internal protection. Botnets, denial-of-service traffic, scanning activity, malicious connection bursts, and compromised devices can therefore affect a distributed network quickly. For an intrusion detection system, the challenge is to identify these behaviors without creating a second privacy problem by centralizing all of the traffic that is being monitored.

A traditional centralized IDS typically transfers network logs or telemetry to a cloud or data-center service for training and analysis. This arrangement offers a convenient global view, but the uploaded data may describe private user routines, device behavior, communication patterns, or sensitive operational activity. Continuous telemetry collection also creates bandwidth cost and a single point of dependence. If connectivity to the central service becomes unavailable,

local networks may temporarily lose part of their detection capability. These limitations are especially important for IoT environments where many low-resource nodes continuously generate small but frequent observations.

Federated Learning (FL) provides a different organization of the learning process. The server distributes a current model to clients, local training is performed on each client's private partition, and only model updates are returned. The server aggregates the received parameters and redistributes the updated global model. Raw records can therefore remain local. However, the literature reviewed for PRIVATE SHIELD also shows that FL does not automatically solve privacy or security. Model updates may leak information, malicious clients may poison the global model, free riders may receive the shared model without genuine training, and heterogeneous non-IID data can make convergence unstable or reduce minority-attack detection [1]–[12].

PRIVATE SHIELD was proposed to study these problems together rather than as isolated algorithms. The project combines local traffic processing, federated training, gradient clipping and differential privacy, suspicious-update validation, client-reliability mechanisms, research-based aggregation and anomaly-detection modules, and an SOC-style dashboard. The goal is not to claim a production-ready security appliance. Instead, the system is designed as a reproducible academic prototype in which privacy, adversarial robustness, heterogeneous client behavior, and human-readable output can be evaluated in one codebase.

II. PROBLEM STATEMENT AND MOTIVATION

The central project problem is the tension between threat visibility and data privacy in distributed IoT environments. A centralized detector gains visibility by collecting large amounts of raw information, but this exposes the same telemetry that the security system is supposed to protect. Moving the training process to the edge reduces raw-data transfer, yet distributed learning creates a new attack surface around the exchanged model updates. A useful solution must therefore protect both the original data and the integrity of the collaborative learning process.

The uploaded literature review identified six connected concerns. First, centralized raw-data collection can cause privacy exposure and heavy bandwidth consumption. Second, gradients and shared parameters may still reveal private information even when raw records never leave the client. Third, a compromised participant may submit scaled, reversed, randomized, or otherwise manipulated updates that degrade the global model. Fourth, a free-rider client may consume the global model while performing little or no genuine local training. Fifth, IoT clients naturally observe non-IID traffic because cameras, sensors, plugs, gateways, and

other device types generate different normal and attack distributions. Sixth, many research systems focus on accuracy and algorithmic behavior but provide limited support for translating low-level model events into readable security explanations.

These concerns motivate a layered design. Privacy protection and update-integrity checks must coexist because they address different risks. Non-IID experiments are necessary because an IID split would hide the diversity of real IoT clients. Client-reliability signals are needed because a mathematically valid update is not necessarily evidence of meaningful participation. Finally, an operator interface is useful because security decisions cannot rely on accuracy tables alone. The project therefore treats the IDS, federated optimizer, adversarial tests, reliability layer, and dashboard as parts of one experimental workflow.

III. OBJECTIVES, SCOPE, AND EXPECTED CONTRIBUTIONS

A. Objectives

- Build a decentralized training pipeline in which local IoT traffic stays with the client and only model parameters or limited metadata are exchanged.
- Support lightweight local models and deliberately non-IID client partitions to represent heterogeneous device behavior.
- Add gradient clipping and differential privacy as a privacy-preserving layer for shared updates.
- Evaluate poisoning-oriented update validation and client-reliability mechanisms before or during aggregation.
- Implement research-based baselines for dynamic aggregation, free-rider defense, and semi-supervised anomaly detection.
- Expose model state, alerts, client participation, and privacy/robustness events through a React-based SOC interface with WebSocket updates and structured LLM-assisted summaries.

B. Scope

The project scope is primarily software-based. The proposal identifies Python, PyTorch, Flower, Opacus, Scikit-learn, FastAPI, MQTT, and a React front end as core technologies. Simulated IoT clients are used for the current experiments, while Raspberry Pi and ESP32-class devices remain part of the planned physical direction. The system does not attempt to fabricate custom hardware, deploy global commercial infrastructure, or claim protection against every adaptive or colluding attacker. This controlled scope allows the work to concentrate on measurable learning, privacy, and robustness behavior.

C. Expected Contributions

The expected contribution is a unified educational prototype rather than a single new mathematical algorithm. The system connects research ideas that are often evaluated separately: privacy-preserving FL, poisoned-update validation, robust or adaptive aggregation, free-rider analysis, semi-supervised anomaly detection, and operator-level visualization. A second contribution is the use of a common evaluation plan so that centralized, standard FL, and research-based variants can be compared under consistent client counts, data splits,

communication rounds, and random seeds. A third contribution is a documented attack and evaluation workflow that future student teams can reproduce or extend.

IV. LITERATURE REVIEW AND RESEARCH GAPS

A. Federated IDS for Heterogeneous IoT

The literature review shows sustained interest in federated intrusion detection because local training reduces the need to centralize raw IoT traffic [1]–[6]. The reviewed methods address heterogeneous environments through dynamic aggregation, adaptive weighting, client clustering, personalization, knowledge distillation, or lightweight models. These mechanisms respond to the fact that IoT clients do not necessarily observe the same features, attack ratios, or normal ranges. A global model built with a simple average can therefore be influenced by clients whose local data are highly skewed or whose local performance is weak for minority attack classes.

FedMADE [6] is particularly relevant to the current implementation because it changes the contribution of local models according to class-level behavior. The project baseline builds a Class Probability Matrix from validation predictions, groups similar models using DBSCAN, and assigns dynamic aggregation weights so that models with stronger class-level contributions have greater influence. This provides a practical contrast to standard FedAvg, where sample count is the main weighting factor. PRIVATE SHIELD uses this idea as a research module for studying non-IID and suspicious-client behavior rather than assuming that every local update should receive the same type of trust.

B. Privacy Preservation in Federated Learning

Local training protects raw records from routine central collection, but the literature warns that gradients and model updates can still reveal information [7], [8], [10]. Differential privacy, secure aggregation, controlled noise, gradient clipping, and proof-oriented mechanisms are common responses. PRIVATE SHIELD adopts the lightweight part of this direction by planning Opacus-based clipping and Gaussian or equivalent privacy noise. The privacy budget is intended to be reported through epsilon and delta so that privacy can be compared against detection utility rather than treated as a binary feature.

The literature also emphasizes a trade-off: stronger privacy mechanisms may increase computation or communication overhead, and larger noise levels can reduce model utility, especially for minority classes. This is important for constrained IoT clients. The project therefore avoids describing privacy as “free.” Planned experiments will vary the privacy setting and measure its effect on accuracy, recall, macro-F1, training time, and communication behavior. The aim is to identify practical settings for the academic prototype, not to assert a universal optimum.

C. Poisoning and Robust Aggregation

A federated server may receive updates from compromised or intentionally malicious clients. Reviewed poisoning work considers corrupted gradients or parameter vectors that can

destabilize global learning [7], [9], [10]. Robust aggregation methods attempt to identify abnormal updates using distance, similarity, audit, or dynamic weighting logic. The proposed PRIVATE SHIELD baseline uses Euclidean or related distance checks around a trusted reference and allows anomalous updates to be rejected or down-weighted before aggregation. More advanced research modules, such as FedMADE and SecureDyn-FL-inspired auditing, are evaluated separately so that their behavior can be compared with ordinary FedAvg.

D. Free-Rider Behavior

Free-rider attacks are different from direct poisoning. A selfish client may download and benefit from the global model while contributing little meaningful local computation [11], [12]. This can reduce fairness and may distort assumptions about client participation. The WEF study [12] motivates the project’s free-rider baseline. Instead of relying only on final accuracy, the defense monitors how frequently selected model weights meaningfully evolve during local training. A client that performs genuine optimization should produce a characteristic pattern of weight changes, while a client that simply copies, reuses, or minimally modifies parameters can produce a different frequency profile.

E. Semi-Supervised Anomaly Detection

FedMSE [13] adds a different perspective by treating intrusion detection as a semi-supervised anomaly problem. Its SAE-CEN structure combines a Shrink Autoencoder with a centroid-based one-class classifier and focuses on learning normal traffic. This is useful in security settings where attack labels may be limited or future attacks are unknown. The MSEAvg aggregation method gives more influence to local models with lower reconstruction error. In PRIVATE SHIELD, this module provides a contrast to supervised classifiers and class-aware aggregation, allowing the project to examine whether normal-only learning behaves differently under non-IID gateway partitions.

F. Resource Constraints and Explainability

The review material also emphasizes the resource limits of IoT devices and the operational gap between model metrics and administrator understanding [14]. Complex cryptography, large deep networks, or high-frequency communication may be unrealistic on low-resource nodes. For this reason, the project favors local processing, parameter sharing, lightweight models, short local epochs, and modular defenses that can be enabled or compared independently. At the operator side, the proposed SOC dashboard displays system state, alerts, privacy settings, client status, and model behavior. Structured event evidence can be passed to an LLM explanation module, while raw local training records and packet payloads remain outside that interface.

G. Rationale for the Selected Research Implementations

The four implemented research directions were selected because together they map closely to the major gaps identified in the literature review. SecureDyn-FL provides a privacy-preserving and robust federated IDS reference and gives the

project an end-to-end federated training structure. FedMADE focuses on dynamic aggregation in heterogeneous IoT environments, which is directly relevant to non-IID client data and unequal attack-class distributions. The WEF method targets free-rider behavior, a threat that is not equivalent to poisoning because the returned update may look numerically ordinary even when genuine local optimization has not occurred. FedMSE adds a semi-supervised anomaly-detection direction and therefore broadens the project beyond fully supervised attack classification.

This selection also supports comparison across different decision signals. FedAvg uses mainly local sample count; FedMADE adds class-level validation behavior and clustering; WEF examines how model weights evolve during local training; FedMSE uses reconstruction error and centroid distance. Studying these signals in one project makes it possible to ask whether a client should be trusted because it owns many samples, because it performs well on specific classes, because it exhibits genuine optimization behavior, or because its normal-traffic model reconstructs data reliably. PRIVATE SHIELD does not assume that one signal is universally superior. Instead, the planned experiments will measure where each mechanism is useful and where it introduces extra complexity.

The selected papers also reflect the project’s staged implementation strategy. The first milestone was to establish a complete basic federated loop. The next milestone was to add working baselines that address heterogeneous aggregation, free-rider defense, and semi-supervised detection. This sequence reduces integration risk because each research idea can first be tested separately. Only after the separate modules are stable will the project combine privacy noise, robust update filtering, reliability scoring, and SOC reporting. The report therefore treats research-paper implementation as a method-selection process rather than as a collection of unrelated reproductions.

TABLE I. RESEARCH GAP TO PROJECT RESPONSE

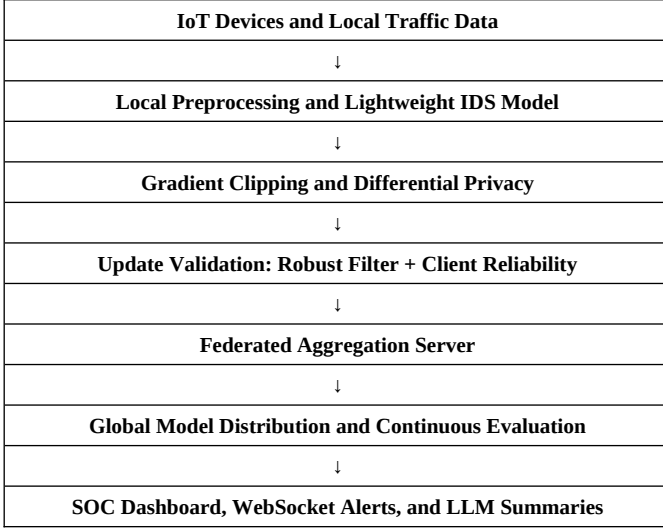
Research gap	PRIVATE SHIELD response
Raw-data exposure / bandwidth	Local training with parameter sharing instead of centralized telemetry collection.
Gradient privacy leakage	Gradient clipping and differential privacy with explicit privacy-utility measurement.
Poisoned updates	Distance/similarity validation, rejection or down-weighting, plus research-based robust aggregation.
Free-rider participation	WEF behavior analysis and reliability/reputation-oriented validation.
Non-IID traffic	Client-local partitions, adaptive aggregation, macro and per-class evaluation.
Weak SOC explainability	Live dashboard, WebSocket alerts, and structured LLM-assisted summaries.

V. PROPOSED SYSTEM ARCHITECTURE

The proposed architecture contains four main layers: distributed IoT clients, local detection and privacy processing, a federated coordination and validation layer, and an SOC interface. Each client receives or captures local traffic,

converts it to structured features, and trains a local detector. The raw client dataset remains local. After training, the update can be clipped and privacy-protected before transmission. The server then validates received updates, applies an aggregation strategy, records client behavior, and redistributes the resulting global model. Operational statistics are sent to the dashboard rather than exposing raw client records.

FIG. 1. PRIVATE SHIELD PROCESSING FLOW



A. Client and Local-Model Layer

Each simulated IoT client stores a separate partition of traffic data. Features may include packet rate, byte count, protocol or port information, connection frequency, message timing, device type, and selected statistical flow indicators. For N-BaIoT-compatible modules, the project aligns feature structures before training. Categorical values are encoded and numerical values are scaled according to the relevant training procedure. Different device profiles and attack proportions are used to create non-IID partitions rather than forcing every client to observe the same distribution.

The proposal allows lightweight classifiers and autoencoder-based detectors. A centralized baseline is useful because it provides a reference trained on combined data. The same or comparable model is then trained through FL so that distributed performance, communication cost, and convergence can be compared fairly. Local epochs and batch sizes are kept modest to avoid creating an unrealistic edge-computation assumption in the simulated environment.

B. Privacy and Validation Layer

Before sharing, model updates can be bounded through gradient clipping so that a single training record or local batch cannot create an unbounded contribution. Differential privacy noise can then be applied through Opacus or an equivalent mechanism. At the server, privacy protection is followed by update validation. A simple prototype can compute the L2 distance between a client update and a trusted reference such as the current global model, a median-like center, or a robust center derived from received updates. Clearly abnormal

vectors can be rejected, while borderline vectors may be down-weighted.

C. Aggregation and Client Reliability

Accepted updates are combined by an aggregation strategy. FedAvg is the baseline, while FedMADE provides class-aware dynamic weighting and FedMSE provides reconstruction-error-based MSEAvg. Client reliability is tracked separately because a client can be statistically unusual without being malicious, and a free rider can be unhelpful without sending a strongly poisoned vector. Reliability signals include accepted and rejected contributions, missing rounds, malformed updates, instability, and WEF-style training behavior. The project treats these signals as evidence for weighting or selection rather than as absolute proof of an attack.

D. SOC and Explanation Layer

The planned SOC interface is a React-based dashboard connected to a FastAPI backend. WebSockets provide live state updates without repeated page refreshes. The interface is intended to show connected devices, federated round number, model metrics, detected attack type, alert severity, current privacy setting, suspicious-update events, and client status. For important events, structured evidence such as packet-rate anomalies, suspicious ports, model-update distance, or client-reliability values can be summarized by an LLM assistant. The proposed design explicitly avoids sending raw packet payloads or local training records to the explanation service.

VI. DETAILED METHODOLOGY

A. Data Preparation and Non-IID Partitioning

The methodology begins by separating data preparation from model training. Each experiment defines a training split, validation split, and test split before federated partitioning. The project uses N-BaIoT-compatible data for several implemented modules, while the broader architecture can also accept simulated MQTT-derived features. Client partitions are created so that different nodes observe different device behaviors or attack ratios. This non-IID structure is important because an IID random split would make the federated problem easier and would not reflect the heterogeneity emphasized in the literature review.

For supervised models, labels represent normal traffic and selected attack classes. For the FedMSE baseline, training is normal-only and the anomaly decision is based on the learned representation and centroid distance. Data-processing decisions are recorded so that the same features and preprocessing order can be reused during evaluation. The report avoids claiming that the selected dataset portions reproduce every detail of the source papers; the purpose is to implement their core ideas in a common academic environment.

B. Federated Training Workflow

At communication round t , the server distributes the current global model to available clients. Client k trains locally for a small number of epochs and returns updated parameters together with limited metadata required by the selected aggregation method. Standard FedAvg weights updates

according to local sample count. Research modules may additionally use class-probability behavior, reconstruction error, client grouping, or reliability evidence. After aggregation, the server evaluates the global model, stores round-level metrics, and sends the new parameters for the next round.

The experiment logger records the round number, selected clients, received updates, rejected updates, training duration, evaluation metrics, and any adversarial event active during the round. This makes it possible to compare normal training with poisoning or free-rider scenarios. It also supports the dashboard, which can display the same structured events without receiving the clients' underlying raw datasets.

C. Differential Privacy

Differential privacy is introduced as a controlled experiment rather than a permanent hidden setting. Per-sample or local gradients are clipped to a maximum norm C , limiting the influence of an individual record. Noise is then added using the privacy engine. The system records epsilon and delta together with global utility metrics. Several noise settings are planned because a stronger privacy setting may reduce reconstruction risk while also lowering attack recall. The key evaluation question is therefore the privacy-utility trade-off under the project's data and computing limits.

D. Poisoning Simulation and Robustness Checks

The threat model includes label flipping, update scaling, model replacement, random update injection, and other controlled parameter mutations described in the proposal. A simple distance-based defense provides the first comparison point: the server measures how far each update is from a trusted reference, applies a configurable or robust threshold, and rejects or reduces the weight of strong outliers. The same attack schedule can then be repeated with FedMADE or SecureDyn-FL-inspired logic to compare whether dynamic or audit-based methods improve stability under the same conditions.

E. Free-Rider Simulation

A free-rider experiment includes clients that receive the current global model but perform little useful local optimization before returning an update. The WEF baseline monitors meaningful changes in selected penultimate-layer weights across local epochs. For each weight, a dynamic threshold determines whether a change is counted. The resulting frequencies form a WEF matrix that describes the local training process. The server compares client matrices using Euclidean distance, cosine similarity, and average evolving frequency to separate normal contributors from suspicious free riders. Separate aggregation outputs can then be prepared for trusted and suspicious groups.

F. Evaluation Metrics

The common evaluation plan uses accuracy, precision, recall, F1-score, macro-F1 or per-class measures for imbalanced tasks, and AUC when the chosen detector supports it. Robustness experiments additionally record attack detection or rejection behavior, false positives against benign

clients, convergence changes, and model degradation. System-level measurements include communication time, round duration, and selected memory or execution observations where they can be measured reliably. Exact values reported by the source papers are not copied as project results; only values produced by the project code will be used in the final comparison.

VII. RESEARCH-PAPER IMPLEMENTATION PROGRESS

A. SecureDyn-FL Basic Prototype

The first research-driven implementation completed in the project was a basic SecureDyn-FL prototype [1]. The implemented baseline includes N-BaIoT-compatible preprocessing, client-wise data division, simulated federated clients, local 1D-CNN training, server-side model averaging, and basic checks under IID and non-IID data settings. The complete baseline flow from local training to a global-model update has been executed successfully. This establishes a reference architecture for later privacy and adversarial modules.

The current version is intentionally described as a basic implementation rather than a full reproduction of SecureDyn-FL. Advanced mechanisms identified in the paper, including GMM and Mahalanobis-distance gradient auditing, encrypted aggregation, personalization, pruning, quantization, and complete poisoning evaluation, were outside the basic version reported in the weekly update. These mechanisms remain candidates for later extension after the project's simpler baselines are compared under a common setup.

B. FedMADE Dynamic Aggregation

The later weekly implementation work added a FedMADE baseline [6]. Local client models are trained independently and evaluated to produce a Class Probability Matrix. DBSCAN is then used to group similar models, and dynamic aggregation weights are calculated so that stronger class-level contributors can have greater influence on the global model. A basic suspicious-client rule is also included so that weak or poisoned-looking updates can receive a very small aggregation weight. This module directly supports the project's study of heterogeneous attack distributions and robust aggregation.

FedMADE is valuable because it exposes a different source of information from standard sample-count weighting. A client can hold many records while still being weak for a particular attack category, and a small client can sometimes provide useful evidence for a minority class. The project does not claim that the baseline exactly reproduces every experimental detail of the paper. Instead, it captures the core dynamic-weighting idea and places it beside other aggregation strategies for controlled comparison.

C. WEF-Defense Against Free Riders

The WEF-Defense implementation [12] focuses on whether a client appears to perform genuine local optimization. The code monitors changes in penultimate-layer weights across local epochs and applies a dynamic threshold to determine

whether each change is meaningful. These counts form a Weight Evolving Frequency matrix for each client. The server compares matrices with Euclidean distance, cosine similarity, and average frequency, allowing normal contributors and suspected free riders to be separated. The latest weekly report records this module as a working baseline whose core client-server and evaluation flow has been checked through local runs.

This module complements poisoning defenses because a free rider may return parameters that are not obviously extreme. Distance from the global model alone may therefore miss the behavior. WEF instead uses evidence from the local training process. In the integrated PRIVATE SHIELD design, such evidence can contribute to a broader client-reliability score rather than acting as the only criterion for exclusion.

D. FedMSE Semi-Supervised Detection

The FedMSE baseline [13] uses N-BaIoT-compatible data and normal-only training. Its SAE-CEN structure combines a Shrink Autoencoder with a centroid-based one-class classifier. The autoencoder learns a compact representation of normal traffic, and centroid distance is used to identify behavior that deviates from the learned normal pattern. During aggregation, MSEAvg gives greater importance to local models with lower reconstruction error. Client datasets are divided into non-IID gateway partitions to represent different devices and traffic conditions.

This implementation provides a useful contrast to supervised attack classification. Rather than requiring every attack category to be present during training, the model learns normal behavior and treats deviation as suspicious. The latest weekly report records the core FedMSE training flow and non-IID client organization as a working baseline, while exact paper-level performance is not claimed because the project uses selected data portions and the available computing environment.

TABLE II. CURRENT RESEARCH IMPLEMENTATION STATUS

Module / focus	Current project status
SecureDyn-FL — FL IDS baseline	Basic end-to-end implementation completed; advanced auditing/privacy mechanisms remain future work.
FedMADE — dynamic robust aggregation	Working baseline with CPM, DBSCAN grouping, dynamic weights, and suspicious-client down-weighting.
WEF-Defense — free-rider detection	Working baseline with penultimate-weight monitoring, WEF matrix construction, and similarity/distance comparison.
FedMSE — semi-supervised anomaly detection	Working baseline with SAE-CEN, normal-only learning, centroid distance, MSEAvg, and non-IID gateway partitions.

E. Integration Strategy

The four modules are not treated as four independent final systems. They are experimental baselines that expose complementary behavior. SecureDyn-FL provides an end-to-end federated IDS reference; FedMADE studies class-aware dynamic weighting; WEF studies contribution behavior and free riders; and FedMSE studies normal-only semi-supervised

detection with reconstruction-error-based aggregation. Integration will proceed only after common experiments identify which components are useful under the same data and client conditions. This avoids combining mechanisms merely because they appeared in different papers.

VIII. THREAT MODEL AND EVALUATION PLAN

A. Security Assumptions

The academic prototype assumes a coordination server that follows the FL protocol but should not learn clients' raw training records. Some clients may be malicious, compromised, inactive, unstable, or selfish. Network services such as MQTT, HTTP, and WebSockets operate in a controlled environment. A real deployment would additionally require secure transport, authentication, access control, key management, and hardened endpoints. These deployment controls are acknowledged but are not the central experimental contribution of the current software prototype.

B. Attacks Considered

Traffic-level scenarios include denial-of-service activity, port scanning, and Mirai-style connection bursts from the earlier system plan. Federated-level scenarios include label flipping, random or scaled updates, model replacement, and free-rider behavior. Gradient-inference risk is addressed conceptually by the privacy layer and remains part of the broader roadmap. The framework does not claim complete resistance to colluding clients, adaptive attackers that know every defense threshold, or a fully compromised coordination server.

C. Common Experimental Setup

A fair comparison requires the same client count, data split policy, training/test partitions, random seed, local epoch budget, and number of communication rounds wherever the compared methods allow it. This common setup is one of the explicit future objectives from the weekly implementation report. FedAvg will serve as the principal federated baseline, while centralized training provides a non-federated reference. Research variants will then be enabled one at a time so that differences in accuracy, convergence, privacy, and robustness can be attributed to a specific mechanism.

TABLE III. PLANNED EVALUATION DIMENSIONS

Dimension	Measurements / comparisons
Detection utility	Accuracy, precision, recall, F1, macro/per-class metrics, AUC when applicable.
Federated behavior	Round-level convergence, IID vs non-IID behavior, selected-client and global metrics.
Poisoning robustness	Model degradation, rejected malicious updates, benign false-rejection behavior.
Free-rider robustness	Detection/separation behavior under different free-rider ratios and client mixes.
Privacy	Epsilon/delta with clipping/noise settings and resulting utility trade-off.
System cost	Communication time, round duration, and available execution or resource observations.

D. Tools and Technologies

The project proposal identifies Python as the main programming language, PyTorch for neural models, Flower for federated orchestration, Opacus for differential privacy experiments, and Scikit-learn for classical models and supporting evaluation. Mosquitto MQTT is used in the broader IoT simulation pipeline. FastAPI provides the planned asynchronous backend, with React for the SOC interface and WebSockets for live updates. Additional proposal components include PostgreSQL, InfluxDB, Redis, Docker, D3.js, Recharts, and an LLM API for structured forensic summaries. Not every listed technology is required for every experiment; the research baselines remain modular so they can be tested without the complete dashboard stack.

E. Project Timeline and Current Position

The original project plan moves from MQTT and traffic ingestion through centralized baselines, federated learning, non-IID stress testing, differential privacy, adversarial simulation, dashboard integration, and final validation. The current implementation work has already produced a basic SecureDyn-FL reference and working baselines for FedMADE, WEF-Defense, and FedMSE. The next project stage is therefore not simply to add more papers. It is to standardize the experimental setup, complete the privacy and adversarial layers, compare the implemented modules, and integrate only the strongest or most compatible components into the main PRIVATE SHIELD pipeline.

F. Reproducibility and Reporting Protocol

Reproducibility is important because the implemented methods were originally evaluated under different assumptions. The project will therefore record the dataset portion, feature set, preprocessing sequence, client partition rule, random seed, local epoch count, batch configuration, client-participation rule, communication-round count, aggregation method, and enabled attack or defense for each experiment. When a method requires a different model structure, that difference will be stated instead of forcing an artificial one-model comparison. Repeated runs can then separate normal training variation from changes caused by a defense or adversarial client.

Results will be reported as project-generated measurements only. Values quoted in the literature review are evidence about prior work, not targets that the implementation must reproduce exactly. This is particularly important because the weekly reports state that selected dataset portions and the team’s available computing environment differ from the full conditions used in the papers. If a baseline achieves a lower or higher value than a source paper, the project report will describe the local configuration and measured outcome rather than silently replacing it with the published number.

The same reporting rule applies to robustness. A defense should not be judged only by whether global accuracy remains high. The evaluation should also record whether malicious updates were actually rejected, whether benign clients were incorrectly penalized, how many rounds were required to converge, and whether the defense adds noticeable training or

communication cost. For free-rider experiments, the client mixture and free-rider ratio must be stated. For differential privacy, epsilon and delta must accompany utility metrics. These records make the final comparison more transparent and consistent with the project’s research-oriented scope.

IX. LIMITATIONS AND FUTURE WORK

A. Limitations

Several limitations are kept explicit. Simulated clients cannot reproduce every timing, power, memory, radio, and hardware constraint of real IoT devices. Selected portions of public data may not reproduce the exact experimental conditions of the research papers. A lightweight Euclidean-distance defense can miss carefully crafted adaptive poisoning. Differential privacy may reduce recall for rare attacks. WEF or reputation signals can be influenced by unstable clients and should not be interpreted as definitive proof of malicious intent. Semi-supervised anomaly detectors can flag unusual but benign behavior. Finally, LLM-generated summaries may be incomplete or incorrect and must be presented together with the underlying structured evidence.

The project also distinguishes between implementation and reproduction. A working baseline means that the core method executes in the project environment; it does not mean that every architecture detail, dataset split, client count, hyperparameter, hardware assumption, or paper-reported metric has been reproduced. This distinction is important for an academic report because it prevents the project from presenting literature values as experimental results generated by the team.

B. Future Work

- Run SecureDyn-FL, FedMADE, WEF-Defense, FedMSE, FedAvg, and centralized baselines under a documented common setup where method assumptions permit fair comparison.
- Complete the differential-privacy pipeline, record epsilon/delta, and measure the privacy-utility trade-off across several noise settings.
- Expand poisoning experiments with consistent malicious-client ratios and compare standard averaging, dynamic weighting, and update-validation defenses.
- Test WEF-Defense under multiple free-rider ratios and study how training-behavior signals can be combined with a recoverable reputation score.
- Evaluate FedMSE beside supervised detectors to compare normal-only anomaly learning with labeled attack classification under non-IID data.
- Integrate the selected mechanisms into the FastAPI/React SOC pipeline and display round-level evidence, rejected updates, client state, and model metrics.
- Move selected experiments toward Raspberry Pi or ESP32-class edge hardware after the software pipeline is stable, while measuring the additional resource constraints explicitly.

X. CONCLUSION

PRIVATE SHIELD is a privacy-preserving and adversarial-aware IoT intrusion-detection project built around the idea that raw telemetry should not need to leave every client in order to

create shared threat intelligence. The literature review showed that federated learning is promising but incomplete by itself: gradients may leak information, poisoned clients can damage the global model, free riders can benefit without genuine contribution, non-IID traffic complicates aggregation, and purely technical metrics provide limited help to security operators. The proposed architecture therefore combines local learning, differential privacy, update validation, research-based aggregation and reliability modules, continuous evaluation, and a live SOC interface.

The project has progressed from proposal and literature review into research-driven implementation. A basic SecureDyn-FL federated IDS prototype is complete, while FedMADE, WEF-Defense, and FedMSE now provide working baselines for dynamic aggregation, free-rider analysis, and semi-supervised anomaly detection. The next goal is a controlled comparison rather than unsupported performance claims. By evaluating the modules under consistent data, client, and adversarial settings, the project can identify which combination provides the most useful balance among detection utility, privacy, robustness, client reliability, communication cost, and operational clarity.

REFERENCES

- [1] I. A. Soomro, H. U. Rehman, S. J. Hussain, A. Iqbal, W. Khalid, and H. Yu, "SecureDyn-FL: A Robust Privacy-Preserving Federated Learning Framework for Intrusion Detection in IoT Networks," arXiv:2601.06466, 2026.
- [2] S. Izadi and M. Ahmadi, "Adaptive Meta-Aggregation Federated Learning for Intrusion Detection in Heterogeneous Internet of Things," arXiv:2602.12541, 2026.
- [3] S. Izadi and M. Ahmadi, "Lightweight Cluster-Based Federated Learning for Intrusion Detection in Heterogeneous IoT Networks," arXiv:2602.12543, 2026.
- [4] G. Singh, K. Sood, P. Rajalakshmi, and Y. Xiang, "Sentinel: Dynamic Knowledge Distillation for Personalized Federated Intrusion Detection in Heterogeneous IoT Networks," arXiv:2510.23019, 2025.
- [5] J. Shen, W. Yang, Z. Chu, J. Fan, D. Niyato, and K.-Y. Lam, "Effective Intrusion Detection in Heterogeneous Internet-of-Things Networks via Ensemble Knowledge Distillation-based Federated Learning," arXiv:2401.11968, 2024.
- [6] S. Sun, P. Sharma, K. Nwodo, A. Stavrou, and H. Wang, "FedMADE: Robust Federated Learning for Intrusion Detection in IoT Networks Using a Dynamic Aggregation Method," arXiv:2408.07152, 2024.
- [7] N. Rattanavipanont and I. De Oliveira Nunes, "Poisoning Prevention in Federated Learning and Differential Privacy via Stateful Proofs of Execution," arXiv:2404.06721, 2024.
- [8] A. ElZemity and B. Arief, "Privacy Threats and Countermeasures in Federated Learning for Internet of Things: A Systematic Review," arXiv:2407.18096, 2024.
- [9] E. Mármol Campos, A. González Vidal, J. L. Hernández Ramos, and A. Skarmeta, "FedRDF: A Robust and Dynamic Aggregation Function against Poisoning Attacks in Federated Learning," arXiv:2402.10082, 2024.
- [10] P. Mai, R. Yan, and Y. Pang, "RFLPA: A Robust Federated Learning Framework against Poisoning Attacks with Secure Aggregation," arXiv:2405.15182, 2024.
- [11] J. Wang, X. Chang, J. Mišić, V. B. Mišić, and Y. Wang, "PASS: A Parameter Audit-based Secure and Fair Federated Learning Scheme against Free-Rider Attack," arXiv:2207.07292, 2022.
- [12] J. Chen, M. Li, T. Liu, H. Zheng, Y. Cheng, and C. Lin, "Rethinking the Defense Against Free-rider Attack From the Perspective of Model Weight Evolving Frequency," arXiv:2206.05406, 2022.
- [13] V. T. Nguyen and R. Beuran, "FedMSE: Semi-supervised Federated Learning Approach for IoT Network Intrusion Detection," *Computers & Security*, vol. 151, Art. no. 104337, 2025.
- [14] V. Ieropoulos, E. Anthi, T. Spyridopoulos, P. Burnap, I. Mavromatis, A. Khan, and P. Carnelli, "Collaborative Intrusion Detection in Resource-Constrained IoT Environments: Challenges, Methods, and Future Directions—A Review," *Journal of Information Security and Applications*, vol. 93, Art. no. 104127, 2025.